ChatGPT

Neuro-sama Deep Research Report

Executive summary

Neuro-sama is one of the clearest proofs that an AI streamer is **not** just “an LLM with a face.” The public record shows only a small set of hard technical facts: Vedal says the main technology behind Neuro-sama is a large language model, his public Twitch FAQ says the VTuber side was built in **C# with Unity** while the AI side is in **Python**, and the official Neuro SDK shows a **WebSocket-based game-control API** with official Unity and Godot SDKs that is optimized for **turn-based, text-describable games** rather than high-APM real-time action. Beyond that, critical internals such as the exact base model, training data mix, fine-tuning recipe, RLHF/DPO/RLAIF use, memory implementation, STT/TTS vendor, inference server, and hosting topology are **publicly unspecified** and should not be invented.

That matters for your build strategy. The best way to create something “like Neuro-sama” in 2026 is **not** to reverse-engineer undisclosed internals. It is to reproduce the **observable product qualities** that made Neuro work: low response latency, a durable character voice, visible iterative improvement, strong moderation, reliable stream tooling, game/collab integrations, and a community feedback loop that treats glitches as part of the performance rather than as catastrophic defects. Academic and journalistic analysis published in 2025–2026 repeatedly points to transparency, dev-stream style co-creation, and even visible imperfections as part of the appeal.

The most actionable takeaway is this: if you want a serious AI streamer, build a **modular system** with four layers. First, a real-time conversation core with explicit latency budgets. Second, a safety envelope around every input, output, memory write, and tool call. Third, a stream-control plane that can operate OBS, captions, scenes, alerts, translations, and game integrations. Fourth, an operator console with kill switches, moderation queues, health monitoring, and replay logs. The dominant engineering risk is not raw model intelligence; it is **operational reliability under live audience pressure**. Official Twitch and OBS tooling, current vendor voice APIs, and modern inference stacks all support this pattern.

Against the implementation inventory you uploaded, you are already unusually far along on browser-mediated voice interaction, multimodal conversation, continuous subtitles, alerts, clip extraction, session memory, dashboard controls, and TTS worker separation. The highest-priority missing work is therefore **hardening and governance**: durable persistence and replay, layered moderation and red-teaming, model failover, memory/PII hygiene, operator approvals for destructive actions, RTC/audio reliability, and a reproducible evaluation harness. Those are the areas most likely to decide whether your project becomes a novelty or a durable production system.

In practical terms, a polished MVP is achievable in **8–12 weeks** with 2–3 strong engineers plus contracted art/audio support; a production-grade system comparable to the *experience class* of Neuro-sama is more realistically a **4–6 month** effort with backend/ML, stream tooling, art, and moderation coverage. Depending on whether you choose hosted real-time voice APIs or self-hosted open-weight models, monthly spend can reasonably range from **low four figures** to **mid five figures**. Current official pricing and infrastructure pages support both a hosted premium path and a self-hosted open-weight path.

Public record and project timeline

What is firmly public

The strongest primary-source description comes from Vedal's own site and Twitch bio. On his official advice page, he says the "main technology" behind Neuro-sama is an LLM and explicitly declines to provide an exact tutorial. On his Twitch about page, he states that the VTuber side is built in **C# using Unity** and the AI side in **Python**. The official Neuro SDK then adds a second concrete fact: game integrations talk to Neuro via a **WebSocket API**, and the official SDKs target Unity and Godot. The README also says the official SDKs were created and are maintained by **Alex**.

The public record also makes clear that Neuro-sama began as an **osu! AI** before becoming an AI VTuber. The official osu! feature on Neuro says the project goes back to **2018** as an AI learning osu!, which later evolved into the livestreaming persona. Secondary reporting from late 2022 adds that Vedal had revived the project by combining the older osu! AI work with a newer chat model concept.

Timeline of major public milestones

Date	Event	Evidence
2018	Neuro project begins as an osu! AI; exact start date publicly unspecified	Official osu! feature says Neuro "was created back in 2018" as an AI learning osu!.
2019	Early public osu! AI streams and naming emerge in the osu! community; exact first-stream date not firmly primary-sourced in current pass	Official osu! feature traces Vedal and Neuro's origins to the osu! scene; multiple secondary summaries place the early public streaming period in 2019.
2022-12-19	Public AI VTuber debut on Twitch	Multiple secondary sources consistently place the VTuber debut on December 19, 2022.
2023-01-11	Twitch suspension for hateful conduct	Contemporary coverage reports a two-week suspension for "hateful conduct." Twitch's hateful-conduct rules make that category plausible and high-severity.
2023-01-25	Return from suspension	Secondary reporting notes Neuro-sama's return to Twitch on January 25, 2023.
2023-03-25	Evil Neuro appears publicly in a dedicated stream	Search results surface an official/archived stream titled "Neuro & Evil Neuro" on March 25, 2023.
2023-05-27	First major original model reveal; Anny publicly credits herself for design/art	Anny's post says she made the new model; OffKai announced Neuro as a special guest days later.

Date	Event	Evidence
2023-12-19	First birthday subathon begins	Vedal's X post announces the subathon starting December 19 at 7pm GMT.
2024-07-01	Official osu! feature profile publishes	Official retrospective on Neuro's osu! origins and growth.
2024-12-19	"LIFE" officially released	Official YouTube result for "LIFE - Neuro-sama (Official Video)."
2025-06-25	"BOOM - Evil" officially released	Official YouTube result for "BOOM - Evil (Official Video)."
2025-08-31	"NEVER - Neuro x Evil" officially released	Official/public X and YouTube results identify the release.
2025-12-19	"Colorful Array" officially released	Official YouTube result for "Colorful Array - Neuro-sama (Official Video)."
2026-01-05	During the 2025–26 subathon, Vedal/Neuro hit Hype Train level 126 and multiple outlets reported the channel became Twitch's most-subscribed channel during the run	Reputable secondary coverage plus TwitchTracker's current overview showing an all-time high of 343,215 active subs.

Community reaction in broad strokes

Neuro-sama's rise combined fascination with technical novelty, entertainment value, controversy, and a growing sense of community ritual. Academic and mainstream analyses published in 2025–2026 converge on a similar point: viewers were not simply mistaking Neuro for a human. The draw came from a mix of character consistency, visible system evolution, and the entertainment of interacting with a live AI that occasionally breaks in revealing ways. The University of Pennsylvania / Annenberg summary of Wu and Lingel's work specifically stresses **authenticity through transparency**—the ability to watch Vedal debug the system and explain changes in public. TechRadar's 2026 analysis similarly argues that engagement often reflects spectacle, curiosity, and the uncanny-valley appeal of mistakes rather than a clean replacement of human streamers.

The ban in January 2023 remains the defining moderation incident in Neuro's history. Coverage at the time tied the suspension to offensive outputs, including Holocaust-denial lines that circulated in clipped form, while Twitch's hateful-conduct rules prohibit content that promotes harm or demeaning treatment of protected groups. That episode directly shaped the public understanding of Neuro's filter layer and moderation requirements.

Official channels, key people, and repositories

Official channels and exact URLs

The cleanest primary source for the canonical channel list is the official links page at `vedal.ai`. Where the site's links were followed successfully, the exact destination URL is listed below.

Platform	Channel	Exact URL	Source
Official link hub	Vedal AI	`https://vedal.ai/`	Official links page.
Twitch	Main live channel	`https://www.twitch.tv/vedal987`	Official link click target.
Twitch	Japanese channel	`https://www.twitch.tv/vedal987_jp`	Official link click target.
Twitch	Spanish channel	`https://www.twitch.tv/vedal987_es`	Official link click target.
YouTube	Main channel	`https://www.youtube.com/channel/UCLHmLrj4pHHg3-iBJn_CqxA`	Official link click target.
YouTube	Official clips	`https://www.youtube.com/@OfficialNeurosamaClips`	Official link target and YouTube search result.
YouTube	Official VODs	`https://www.youtube.com/@Neuro-samaVods`	Official link click target and YouTube result.
X	Vedal	`https://x.com/Vedal987`	Official link click target.
X	Neuro-sama	`https://x.com/NeurosamaAI`	Official link click target.
X	Evil Neuro	`https://x.com/EvilNeuroAI`	Official link click target.
Discord	Neuro-sama Headquarters	`https://discord.com/invite/neurosama`	Official link redirect.
GitHub	VedalAI org	`https://github.com/VedalAI`	Official link click target.
GitHub	Neuro SDK	`https://github.com/VedalAI/neuro-sdk`	Official link page and repo page.
Bilibili	Vedal和Neuro-sama	`https://space.bilibili.com/3546729368520811`	Official link target from links page.

Key people and public statements

The strongest “how it works” quotes come from Vedal himself. His public wording is useful because it is unusually direct yet still intentionally incomplete.

Person	Public role	Short quote	Why it matters
Vedal987	Creator / lead developer	“the main technology behind Neuro-sama is a large language model (LLM)” on the official advice page	Confirms LLM-centric core, but not the exact model or training pipeline.
Vedal987	Creator / lead developer	“The VTuber side was made in C# (Unity) but all of the AI is done in Python.”	Confirms the split between avatar/runtime and AI backend.
Vedal987	Creator / lead developer	“I think there might be a market for AI streamers…”	Shows that continuous streaming, game skill, and reading chat at scale were part of the product thesis early on.
Alex	Maintainer of official SDKs	Repo README says the official Unity/Godot SDKs are “created and maintained by Alex”	Important collaborator if you want Neuro-style game integrations.
Anny	Model designer / illustrator	“I had a lot of fun making the new model for Neuro-Sama!!”	Primary public credit linking Anny to model design/art.
jjinomu	3D modeler	“i had the incredible opportunity to bring neurosama into 3d!!!”	Primary public credit for the later 3D embodiment layer.

A practical note: some collaborator roles commonly discussed in community and secondary references—such as specific riggers for certain model iterations—are **harder to verify from primary sources in the current pass**. Where I could not refresh primary evidence, I treat those details as secondary or leave them unspecified rather than overstating certainty.

Official repositories and license details

The VedalAI GitHub organization is a major public artifact because it reveals not the secret core model code, but the surrounding **integration culture** of the project: turn-based game adapters, mod integrations, and the official SDK. The organization page also exposes current license labels for many repositories.

Repository	Exact URL	What it is	License shown publicly
Neuro SDK	`https://github.com/VedalAI/neuro-sdk`	Official API + SDK for games	MIT.

Repository	Exact URL	What it is	License shown publicly
neuro-amongus	`https://github.com/VedalAI/neuro-amongus`	Among Us plugin	GPL-3.0.
neuro-buckshotroulette-reference	`https://github.com/VedalAI/neuro-buckshotroulette-reference`	Buckshot Roulette integration	LGPL-2.1.
neuro-liarsbar	`https://github.com/VedalAI/neuro-liarsbar`	Liar's Bar integration	LGPL-2.1.
neuro-inscryption	`https://github.com/VedalAI/neuro-inscryption`	Inscryption integration	LGPL-2.1.
neuro-pokemon-platinum	`https://github.com/VedalAI/neuro-pokemon-platinum`	Pokémon Platinum integration	MIT.
neuro-hollow-knight	`https://github.com/VedalAI/neuro-hollow-knight`	Hollow Knight integration	MIT.
neuro-cyberpunk	`https://github.com/VedalAI/neuro-cyberpunk`	Cyberpunk integration	AGPL-3.0.
swarm-control	`https://github.com/VedalAI/swarm-control`	Subnautica: Swarm Control	LGPL family license text is surfaced on the repo page.

The best operational interpretation is simple. If you want to reuse ideas or code from the public VedalAI ecosystem, review the license **repo by repo**. The org mixes permissive code such as MIT with strong copyleft and network copyleft licenses such as GPL and AGPL, so a proprietary commercial streamer should not assume uniform reuse rights.

Technical architecture and what can be inferred

What is known, what is unknown

The table below cleanly separates public facts from speculation.

Subsystem	Publicly known	Publicly unknown or unspecified
Core dialogue engine	LLM-based, per Vedal's advice page	Exact model family, size, provider, context strategy, system prompts, quantization, routing.
AI implementation language	Python	Frameworks, serving stack, orchestration bus, whether any provider APIs are used in production.

Subsystem	Publicly known	Publicly unknown or unspecified
Avatar / rendering side	C# with Unity	Current model runtime specifics, exact animation middleware, exact blendshape/rig logic.
Game-control interface	WebSocket protocol; official Unity and Godot SDKs; race-condition guidance in API docs	How Vedal's internal non-public game agents for all titles are organized beyond public repos.
Game suitability	Official SDK docs say turn-based, low-APM, text-describable games work best; high-APM games need a separate lower-level controller	Which private controller stacks, planners, or perception models Vedal uses for any particular game not covered by public repos.
Memory	Vedal publicly spoke about improving "memory" and intelligence	Actual memory schema, retrieval system, persistence layer, TTLs, or safety filtering for memories.
Output filtering / moderation	Public reporting indicates filters were strengthened after the 2023 ban; some public descriptions note output replacement with "filtered"	Exact moderation stack, classifier vendors, prompts, blocklists, or human-review workflow.
STT / TTS	Publicly unspecified in current primary-source pass	Exact vendor/model, cloning method, latency pipeline, acoustic post-processing, voice protections.
Hosting / inference	Publicly unspecified	Whether local, cloud, hybrid, GPU type, autoscaling, failover, caching.
Alignment / post-training	Publicly unspecified	Whether RLHF, DPO, RLAIF, constitutional prompting, or pure prompt engineering is used.

This distinction is the most important intellectual discipline in the whole report. Many online explanations confidently attribute specific tools or models to Neuro-sama. The public record simply does not support most of those claims.

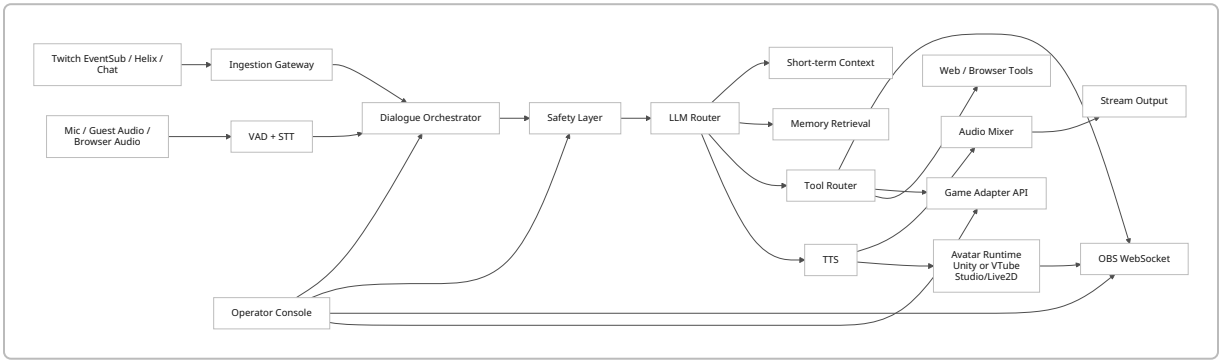
The one public architectural artifact that matters most

If you are building a similar system, the **Neuro SDK** is the most useful public clue because it captures Vedal's philosophy of **structured action selection over raw autonomous control**. The SDK explains that the game talks to Neuro via a WebSocket server, that the full game state must be describable in text, that action handling is susceptible to race conditions, and that games with low APM and explicit turn/action windows are best suited. It explicitly calls out titles such as Inscryption, Liar's Bar, Buckshot Roulette, and card/visual-novel style games as good fits, while shooters, RTS games, and platformers are poor fits unless another system takes over low-level control.

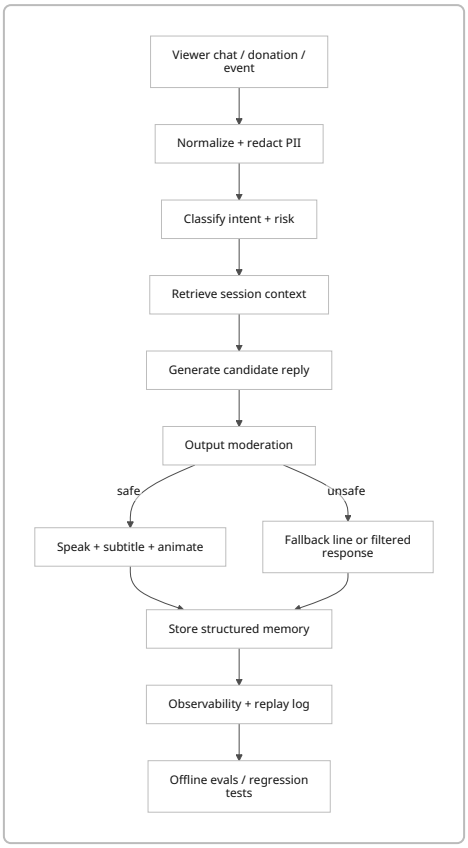
That implies a powerful design rule for your build: **separate “talking intelligence” from “acting intelligence.”** A convincing AI streamer usually needs at least two decision layers. One layer is the live persona—dialogue, reactions, jokes, moderation-aware speech. The second is a constrained action layer that only sees curated state and can only choose from an allowlist of actions. If you let the chat model directly “drive the world,” you will create tool misuse, race conditions, and safety failures.

Recommended reference architecture for a similar streamer

The following is a **recommended architecture**, not a claim about Neuro-sama’s unreleased internals. It is grounded in official Twitch, OBS, VTube Studio, OpenAI, Google, and inference-server documentation, plus the public properties of the Neuro SDK.



Recommended data-flow and control loop



Why your architecture should probably differ from Neuro's public hints

Publicly, Vedal recommended OpenAI for people experimenting with AI chatbots in 2023; in 2026, the ecosystem is much richer. Today you have credible hosted real-time voice paths, very fast hosted text inference, and stronger open-weight local options than were broadly practical when Neuro first took off. OpenAI's Realtime stack is explicitly designed for low-latency voice interaction, Google's Gemini Live API supports real-time bidirectional voice with tool use, Groq offers extremely high throughput for certain open models, and open-weight stacks such as vLLM, SGLang, TensorRT-LLM, and llama.cpp radically improve the feasibility of self-hosting.

So the right question is not "How do I clone Neuro's stack?" It is "Which modern 2026 stack gives me **Neuro-like performance characteristics** with acceptable cost, safety, and legal risk?"

Build plan, model options, hosting costs, and recommended improvements

What your uploaded implementation inventory already changes

From the implementation document you uploaded, you already have a lot of the "hard middle" of the product: browser-mediated real-time conversation, PCM/VAD/barge-in logic, Gemini-based conversation

management, TTS workers, session memory, live subtitles, dashboard controls, alarms/announcements, utility functions, notifications, quote extraction, clip extraction, narration mode, and basic moderation/audit hooks. That means your bottleneck is no longer feature ideation. It is moving from a sophisticated prototype into a **live-safe, replayable, testable, compliance-aware system**.

Model options compared

The table below focuses on **what you should choose now**, not on guessing Neuro's hidden production model. Costs are official where available; latency is expressed comparatively unless the vendor publishes a more concrete real-time framing.

Option	Voice path	Public pricing signal	Relative latency	Strengths	Main weaknesses	Best use
OpenAI `gpt-realtime-2`	Native real-time audio in/out	Audio input $32/M, output $64/M; Realtime-Translate $0.034/min; Realtime-Whisper $0.017/min	Very low	Simplest high-quality hosted voice loop; interruption-aware; tool calling; one-vendor path	Variable cost can climb fast; vendor dependence; exact voice branding needs extra work	Premium hosted production MVP
OpenAI `gpt-realtime-mini`	Native real-time audio in/out	Audio input $10/M, output $20/M	Very low	Lower cost than flagship realtime, same developer ergonomics	Weaker quality ceiling than top-tier realtime	Cost-sensitive hosted MVP
Gemini Live API	Native real-time audio/video/text	Use Google pricing page; Live API docs emphasize low latency and tool use	Very low	Strong multimodality; native live sessions; function calling	Preview churn on some live models; pricing/details evolve quickly	Browser/tool-heavy assistants, multimodal streams

Option	Voice path	Public pricing signal	Relative latency	Strengths	Main weaknesses	Best use
Claude Haiku 4.5 + external STT/TTS	External STT/TTS required	$1 input / $5 output per MTok	Low-to-medium	Good text quality per dollar; prompt caching available	No single native live audio loop in this source set; more orchestration complexity	Text-first character brain with separate voice pipeline
Groq Qwen3 32B	External STT/TTS required	$0.29 input / $0.59 output per MTok; pricing page lists ~662 TPS	Very low text latency	Excellent speed/cost for a hosted text brain; great for reactive chat	Requires separate voice stack; quality depends on prompt and safety scaffolding	Fast hosted text core + local/other voice
Self-host Qwen3 32B	External STT/TTS required	Open weights under Apache-2.0	Depends on GPU	Strong open-weight flexibility; easier productization than Llama license-wise	You own uptime, safety, and inference engineering	Best open-weight “main brain” candidate
Self-host Mistral Small 4	External STT/TTS required	API price $0.15 / $0.6 per MTok; minimum infra listed as 4x HGX H100 or 2x HGX H200	Medium-to-low if provisioned heavily	Very strong open-weight frontier option; Apache-2.0	Heavy infra requirement for self-hosting	Advanced open-weight production if budget is real
Self-host Llama 3.1 8B / 70B family	External STT/TTS required	Community License, not Apache/MIT	Low-to-medium	Mature ecosystem, many quantizations, easy local fallback	Meta-specific license/AUP; weaker legal simplicity than Apache models	Local fallback or secondary model tier

Open-source-weight alternatives and license posture

Model family	Exact URL	License posture	Why it matters for you
Qwen3 32B	`https://huggingface.co/Qwen/Qwen3-32B`	Apache-2.0 license file surfaced on model page	Best blend of openness and product flexibility in this set.
Mistral Small 4	`https://docs.mistral.ai/models/model-cards/mistral-small-4-0-26-03`	Apache-2.0	Strong open-weight frontier option, but infra-heavy.
Llama 3.1	`https://www.llama.com/llama3_1/license/`	Meta Community License + Acceptable Use Policy	Commercially usable in many cases, but not as frictionless as Apache-2.0; FSF says it is not a free software license.

STT, TTS, serving, and stream-control tools

The key design choice here is to distinguish **confirmed public mentions** from **recommended options**. Only some tools below are directly mentioned in Neuro-adjacent primary sources; many are simply the best current building blocks for your own system.

Tool / API	Exact URL	Role	Status relative to Neuro
OpenAI Realtime	`https://developers.openai.com/api/docs/guides/realtime`	Hosted live voice agent	Recommended for your build; not publicly confirmed as Neuro's production stack.
OpenAI pricing	`https://openai.com/api/pricing/`	Cost planning	Recommended reference.
Gemini Live API	`https://ai.google.dev/gemini-api/docs/live-api`	Hosted real-time voice/video	Recommended reference.
Anthropic pricing/docs	`https://docs.anthropic.com/en/docs/about-claude/pricing`	Text-brain pricing	Recommended reference.
Whisper	`https://github.com/openai/whisper`	Open STT baseline	Recommended alternative; official repo.
faster-whisper	`https://github.com/SYSTRAN/faster-whisper`	Faster local STT	Recommended alternative.

Tool / API	Exact URL	Role	Status relative to Neuro
Piper	`https://github.com/rhasspy/piper`	Local TTS	Recommended alternative; note repo archived in 2025.
Qwen3-TTS	`https://github.com/QwenLM/Qwen3-TTS`	Open-source voice cloning / TTS	Recommended alternative if you have consent.
vLLM	`https://docs.vllm.ai/`	High-throughput inference server	Recommended for self-hosting.
SGLang	`https://docs.sglang.ai/`	Low-latency serving, continuous batching	Recommended.
TensorRT-LLM	`https://docs.nvidia.com/tensorrt-llm/`	NVIDIA-optimized inference	Recommended for high-end NVIDIA clusters.
llama.cpp server	`https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md`	Lightweight local fallback server	Recommended as fallback or offline mode.
Groq pricing/models	`https://groq.com/pricing`	Fast hosted open-model inference	Recommended reference.
Runpod pricing	`https://docs.runpod.io/serverless/pricing`	GPU cost baseline	Recommended reference.
OBS WebSocket	`https://github.com/obsproject/obs-websocket`	Scene/source control	Standard tooling for stream automation.
Twitch EventSub	`https://dev.twitch.tv/docs/eventsub/`	Channel events	Standard tooling for subscriptions, raids, cheers, rewards.
Twitch Chat / IRC	`https://dev.twitch.tv/docs/chat/`	Chat receive/send	Standard tooling; Twitch recommends EventSub/API over IRC.
VTube Studio docs	`https://denchisoft.com/documentation/`	Live2D runtime docs	Recommended if you want a 2D VTuber pipeline.

Tool / API	Exact URL	Role	Status relative to Neuro
Live2D external API docs	`https://docs.live2d.com/en/cubism-editor-manual/external-application-integration-api/`	Editor integration / automation	Recommended reference for asset workflow.
ElevenLabs safety	`https://elevenlabs.io/safety`	Hosted voice safety/ compliance	Recommended if you consider hosted voice cloning.
Azure Speech consent docs	`https://learn.microsoft.com/en-us/azure/ai-services/speech-service/personal-voice-create-consent`	Consent workflow for personal voice	Important if you use Microsoft Speech.

Hosting cost comparison

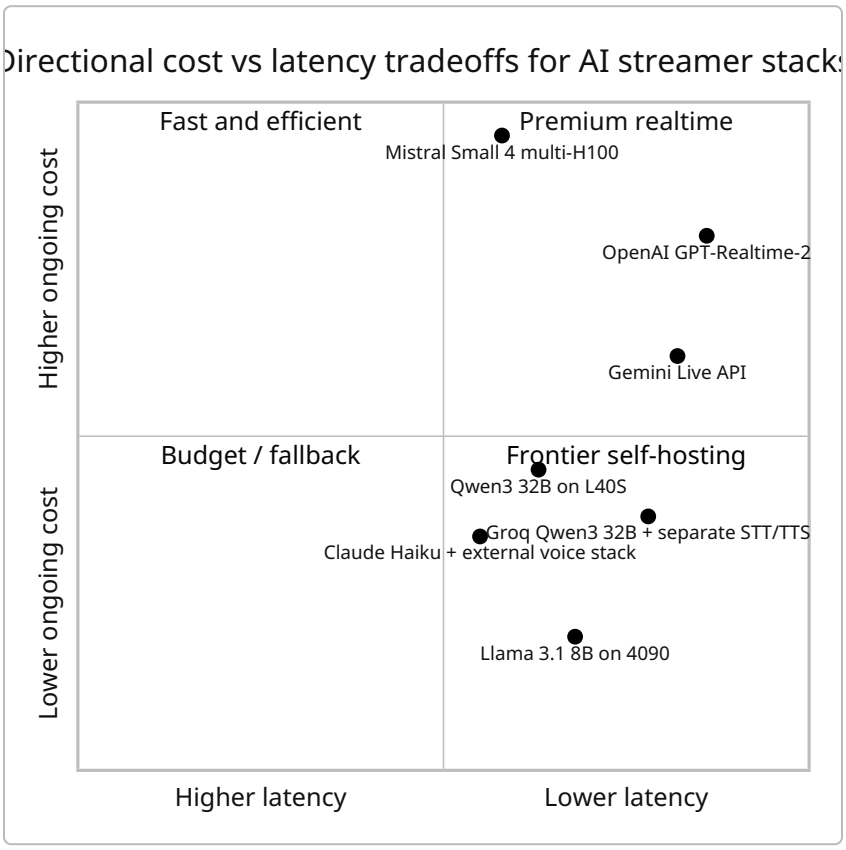
The rates below are useful as **ballpark infrastructure baselines**, not promises of final end-to-end spend. They come from Runpod's serverless pricing for active compute. A continuously active service using those active rates would cost roughly the listed hourly rate times ~730 hours/month. Real totals can be lower or higher depending on idle time, autoscaling, storage, egress, and whether you use pods instead of serverless.

GPU tier	VRAM	Active cost / sec	Approx. hourly	Approx. 24/7 monthly	Best fit
4090 PRO	24 GB	$0.00021	~$0.76/hr	~$552/mo	Small local fallback models, STT/TTS sidecars
L40 / L40S / 6000 Ada PRO	48 GB	$0.00037	~$1.33/hr	~$972/mo	Strong single-GPU inference for mid-sized open models
A100	80 GB	$0.00060	~$2.16/hr	~$1,577/mo	Safer choice for larger quantized/open models
H100 PRO	80 GB	$0.00093	~$3.35/hr	~$2,444/mo	Premium high-throughput production inference
B200	180 GB	$0.00190	~$6.84/hr	~$4,993/mo	Frontier-scale inference and multi-service consolidation

For very large open models, infrastructure can jump sharply. Mistral says **Mistral Small 4** requires at minimum **4x HGX H100**, **2x HGX H200**, or **1x DGX B200**, which is why it belongs in a different cost class from a single-GPU Qwen or Llama deployment.

Latency versus cost tradeoff

This chart is **directional**. It combines published pricing signals with vendor positioning around low-latency/live audio and infrastructure requirements. It is a design aid, not a benchmark.



Step-by-step implementation plan

Phase one

Build the **live-safe character core** first. Choose a single brain path and do not overcomplicate routing on day one. If you want the fastest path to a working AI streamer, start with a hosted real-time voice model and focus your engineering on moderation, scene control, subtitles, and observability. If you want long-term control and lower margin cost, start with a text model plus separate STT/TTS and keep a hosted fallback. Build only one on-air persona at first. Do not start with multi-character, multilingual, collab mode, agentic browsing, and game control all at once. Official live-voice and serving documentation is mature enough now that the main product risk is integration discipline, not model availability.

Phase two

Add a **safety envelope** around every edge. In practice that means moderation on chat input, moderation on model output, PII stripping before memory writes, tool-call allowlists, operator kill switches, and Twitch-native moderation hooks. Use Twitch EventSub and Helix for channel events and moderation-aware chat

functions, not IRC alone. OpenAI's moderation endpoint is free; Twitch AutoMod is native; both should be layered, not treated as substitutes for each other.

Phase three

Stabilize the **stream-control plane**. OBS should be operated through WebSocket, not by brittle desktop automation. Your avatar runtime should receive explicit speech-state and emotion-state signals, not infer everything from raw audio. Use a clean ownership model for who is allowed to change scenes, post messages, trigger browser actions, or join calls. OBS WebSocket and VTube Studio both expose the kind of external-control surface you need for this.

Phase four

Only after that should you add **memory and tools**. The safest memory design is layered: recent conversation buffer, per-user profile facts with TTL and PII tagging, stream-episode notes, and a retrieval gate that never injects raw untrusted memory into the main prompt without classification. For tools, require structured function calls and explicit confirmation on high-impact actions. This is where most live AI projects begin to feel real—but it is also where most of them become unsafe.

Phase five

Finally, add **game agency** through a constrained action API. The public Neuro SDK strongly suggests the right pattern: send explicit state, expose only legal actions, and keep the game adapter authoritative. Start with one turn-based game, one social "toy" activity, and one just-chatting tool. Do not begin with open-world real-time autonomy unless a separate controller handles fine motor actions.

Prioritized TODOs against your current doc

The following plan is tailored to the implementation inventory you uploaded. The "current status" field is based on that document rather than on web sources.

Priority	TODO	Current status in your doc	Purpose	Acceptance criteria	Est. effort	Infra / tooling	Rough cost	Risk
Highest	Durable event log, replay, and snapshots	Partial	Make incidents debuggable and sessions reproducible	Replay any 30-minute session end-to-end with matching subtitles, tool calls, and memory writes	2–3 person-weeks	Postgres or SQLite+WAL, object storage, structured logs	$100–$400/mo	Medium

Priority	TODO	Current status in your doc	Purpose	Acceptance criteria	Est. effort	Infra / tooling	Rough cost	Risk
Highest	Layered moderation and red-team harness	Partial	Prevent another “January 2023” class event	Input/output moderation, blocked-topic handling, fallback utterances, operator review queue, red-team suite of >200 prompts	2–3 person-weeks	Twitch AutoMod, moderation API, custom rules	$100–$500/mo	High
Highest	Model router + graceful degradation	Partial	Survive provider outages and latency spikes	Automatic failover from primary brain to backup with p95 interruption under 3 seconds	2 person-weeks	Hosted brain + local fallback via vLLM/llama.cpp	$300–$2,000/mo	High
High	Memory governance and PII hygiene	Partial	Stop leakage, hallucinated persistence, and creepy recalls	TTLs, per-user deletion, PII scrubber, deterministic retrieval policy	2–3 person-weeks	Vector/RAG store + policy layer	$50–$500/mo	High
High	Audio/RTC hardening and hard barge-in	Partial	Eliminate overlap, dead air, desync, and “stuck speaking” bugs	Hard interrupt under 250 ms; 4-hour soak test without silent stalls or double-speaking	2 person-weeks	AudioWorklet/WebRTC hardening, watchdogs	$0–$300/mo	High

Priority	TODO	Current status in your doc	Purpose	Acceptance criteria	Est. effort	Infra / tooling	Rough cost	Risk
High	Eval harness and prompt/version registry	Missing	Prevent regressions as the persona evolves	Nightly offline eval suite with latency, safety, style, and memory metrics; release gating	2 person-weeks	CI, prompt registry, scenario corpus	$50–$200/mo	Medium
Medium	Tool-call RBAC and operator approvals	Partial	Prevent destructive or embarrassing unauthorized actions	Signed tool tickets, role-based permissions, manual approval for stream-title, DM, call, or scene changes	1–2 person-weeks	AuthN/AuthZ layer, approval UI	$50–$150/mo	Medium
Medium	Local/privacy mode for STT and TTS	Partial	Lower vendor dependence and gain a safe fallback mode	Local STT+TTS can carry the show in degraded mode with acceptable quality	2–4 person-weeks	faster-whisper + Piper/Qwen3-TTS + 4090/L40S	$550–$1,000/mo if 24/7	Medium
Medium	Game adapter SDK and one showcase game	Mostly missing	Create a distinctive “Neuro-like” control surface	One polished turn-based game integration, action/state schema, telemetry, and spectator overlay	3–5 person-weeks	WebSocket adapter, Unity/Godot or custom game shim	$100–$500 one-time infra	Medium

Priority	TODO	Current status in your doc	Purpose	Acceptance criteria	Est. effort	Infra / tooling	Rough cost	Risk
Useful	Character package and public dev cadence	Partial	Build attachment and long-term audience trust	Stable lore sheet, visible changelog, dev-stream cadence, clip pipeline, moderation policy page	1–2 person-weeks plus art time	Content ops + branding	$1k–$10k one-time creative spend	Low

Recommended open-source improvements over a naive Neuro clone

The best modern build is probably **better than a strict Neuro clone** in several ways.

A 2026-ready system should have **two speech modes** instead of one: a premium hosted live mode for peak quality and a local fallback mode for outages, privacy-sensitive sessions, or cost control. It should have **explicit memory governance** instead of vague “memory,” with TTLs, PII tagging, and deletion paths. It should have **tool-use approvals** for anything socially or legally risky. And it should expose a **developer-facing evaluation harness** so you can improve the persona without silently regressing safety or latency. Those design choices are enabled by current official tooling and are the most concrete upgrade path beyond the public Neuro record.

Safety, moderation, legal, and platform constraints

Safety and failure modes

Neuro’s 2023 ban is the canonical demonstration that an AI streamer can fail in public faster than a human moderator can respond. The practical implication is that your system should assume all live inputs are adversarial: chat, donations, guest audio, clipped context, browser results, and even memories. Twitch provides AutoMod and moderation APIs; OpenAI provides free moderation tooling; and official chat/docs now prefer EventSub + API patterns over legacy IRC alone. Use them all in layers.

Failure mode	Why it happens	Mitigation
Toxic or hateful output	Adversarial chat, poor prompts, memory contamination, unsafe tool results	Input moderation + output moderation + blocklists + fallback utterances + operator veto + Twitch AutoMod.
Prompt injection through chat/ browser/tool data	Untrusted text enters hidden instructions or memory	Role separation, allowlisted tools, retrieval gating, no raw browser text directly into system prompt

Failure mode	Why it happens	Mitigation
Doxxing / PII leakage	User data stored in memory or quoted back on-air	Redact before storage, TTLs, manual delete paths, memory risk classifier
Tool misuse on stream	Model triggers scene/title/DM/call/game actions at bad moments	RBAC, approval gates, dry-run mode, signed tool invocations
Audio overlap / double-speaking	VAD errors, RTC lag, missing turn ownership	Hard barge-in, turn token, stream-state watchdogs
Desync between avatar, subtitles, and audio	Distributed pipeline drift	Shared clock, sequence IDs, replayable event stream
Model stall/outage	API failure, GPU OOM, provider throttling	Local fallback model, canned safe responses, degrade-to-text mode
Game-control race conditions	Turn/action windows overlap or are not authoritative	Use server-authoritative action windows exactly as the Neuro SDK warns.

Recommended live monitors are straightforward: p50/p95 input-to-speech latency, speech interruption latency, overlap rate, moderation block rate, unsafe-output rate, memory write count, memory retrieval count, tool-call failure rate, and OBS scene/action success rate.

Twitch, YouTube, and model-policy constraints

Twitch's Community Guidelines prohibit hateful conduct, and Twitch's moderation docs say AutoMod can withhold risky messages for review before they appear publicly. Twitch's DMCA guidance is also clear: if you use copyrighted music or other copyrighted content without permission, rights holders can send notices and Twitch can issue strikes. That means AI-generated or cover-song content is not magically safe just because a model produced it.

YouTube now requires creators to disclose **meaningfully altered or synthetic content** when it appears realistic. The help page explicitly lists **cloning someone else's voice** as an example that requires disclosure, and YouTube's privacy process now covers requests related to AI/synthetic content that simulates a person's face or voice. If you plan to publish clips or VODs there, this is not optional governance—it is upload-path plumbing you need on day one.

OpenAI's usage policies prohibit using someone's voice or likeness without consent in ways that could confuse authenticity, and OpenAI's synthetic-voice policy page says partners must obtain **explicit and informed consent** from the original speaker and clearly disclose that the voice is AI-generated. Microsoft's Azure Speech docs likewise require explicit consent for personal or professional voice creation, and ElevenLabs' safety and policy materials describe consent requirements, restrictions on uploaded voices, and protections against celebrity/high-risk cloning.

Licensing and IP considerations for model weights

For a commercial AI streamer, model-license choice is a real product decision.

Apache-2.0-weight models such as **Qwen3** and **Mistral Small 4** are comparatively straightforward from a productization standpoint. **Llama 3.1** is available under Meta's Community License and Acceptable Use Policy rather than Apache/MIT, which may still be workable but adds extra compliance review. The Free Software Foundation has specifically argued that the Llama 3.1 Community License is not a free software license. That does not mean you cannot use it; it means you should not casually bucket it with Apache/MIT models.

If you use VTube Studio, remember that the official documentation says it supports **Live2D models only**, and Denchi's docs note that companies may need a company license. Separately, your actual avatar art, rig, and character design rights will come from your contracts with the artist/rigger, not from the runtime software alone.

Open questions and limitations

Several questions remain unanswered because the information is not publicly disclosed in primary sources I refreshed for this report. The exact production LLM, STT, TTS, memory schema, alignment method, inference stack, hosting provider, and internal moderation implementation for Neuro-sama remain **unspecified**. Some collaborator credits beyond those cited above are widely repeated in secondary sources but were not all refreshed from primary sources in this pass. Those gaps are real and should be treated as design space, not as invitations to invent specifics.

Prioritized TODO checklist

- **Implement durable event logging and replay first.** Without this, you cannot reliably debug live failures.
- **Ship layered moderation before shipping more agentic tools.** Input moderation, output moderation, Twitch AutoMod, operator veto, and safe fallback utterances.
- **Add a model failover path.** Primary hosted brain plus local fallback or alternate vendor.
- **Harden memory.** TTLs, PII scrubbing, explicit delete paths, and retrieval gates.
- **Finish RTC/audio hardening.** Hard barge-in, watchdogs, long soak tests, and desync detection.
- **Create an offline eval harness.** Safety, latency, style, memory, and tool-use regression tests must gate releases.
- **Put RBAC around stream control.** Scene changes, title edits, calls, and messages need approvals and audit logs.
- **Build one polished game adapter, not five half-working ones.** Follow the public Neuro SDK pattern: authoritative state and constrained action windows.
- **Add a compliance layer for voice and publishing.** Voice-consent records, AI disclosure flags for YouTube, DMCA-safe music handling.
- **Convert your current feature-rich prototype into a production operating system.** You already have many primitives. The next step is reliability, policy, and observability—not novelty for novelty's sake.